

Отчёт по практической работе №8

Аутентификация и авторизация веб-приложений

Выполнил	Аньшин Сергей
Группа	ИСП-406
Дата	2026

2026

Содержание

- 1 Цель и постановка задачи
- 2 Архитектура проекта
- 3 Реализация методов аутентификации
- 4 Сравнение методов
- 5 Меры безопасности
- 6 Проверка работоспособности
- 7 Вывод
- 8 Используемые источники

1 Цель и постановка задачи

Цель работы — реализовать и сравнить основные способы аутентификации и авторизации, которые применяются в современных веб-приложениях. В рамках практики создано единое приложение на Python, позволяющее проверить каждый способ через HTTP-запросы и простой веб-интерфейс.

По заданию требовалось:

- реализовать регистрацию по логину, паролю и электронной почте;
- показать вход с помощью подписанной cookie;
- реализовать серверную сессию с идентификатором в cookie;
- реализовать беспарольный вход по одноразовой ссылке;
- создать многоразовый JWT access token и одноразовый JWT refresh token;
- показать OAuth2 через учебную единую точку входа;
- реализовать токен PASETO v4.local;
- описать достоинства, недостатки и применимость каждого способа;
- подтвердить работоспособность автоматизированной проверкой.

2 Архитектура проекта

Практика выполнена в виде Flask-приложения. PostgreSQL хранит пользователей и серверное состояние одноразовых механизмов. Пароли обрабатываются функциями Werkzeug и сохраняются только как стойкие хеши. JWT создаются библиотекой PyJWT, а PASETO — библиотекой PySETO.

Компонент	Назначение
Flask	HTTP-маршруты, веб-интерфейс и формирование ответов
PostgreSQL	Пользователи, сессии, magic links, refresh-токены и OAuth-коды
Werkzeug	Хеширование и проверка паролей
PyJWT	Access- и refresh-токены стандарта JWT
PySETO	Шифрование и проверка PASETO v4.local

Компонент	Назначение
Docker Compose	Воспроизводимый запуск PostgreSQL

Общая последовательность обработки запроса:

Клиент → Flask-маршрут → проверка учётных данных → PostgreSQL → cookie или токен → защищённый ресурс

3 Реализация методов аутентификации

3.1 Регистрация по логину, паролю и электронной почте

Назначение. Создать учётную запись пользователя и подготовить её для дальнейшей аутентификации.

Как работает. Сервер проверяет длину логина и пароля, формат электронной почты и уникальность значений. Пароль преобразуется в стойкий хеш. Исходный пароль в базе не хранится.

```
cur.execute(
    "INSERT INTO auth_users (login, email, password_hash) "
    "VALUES (%s, %s, %s) RETURNING id, login, email",
    (login, email, generate_password_hash(password)),
)
```

Критерий	Описание
Достоинства	Простая и понятная пользователю схема; не зависит от внешнего поставщика; позволяет управлять политикой паролей.
Недостатки	Необходимо безопасно хранить хеши, реализовать восстановление доступа и защиту от перебора.
Применимость	Подходит для автономных сайтов и информационных систем с собственной базой пользователей.

3.2 Авторизация с помощью подписанной cookie

Назначение. Сохранить состояние входа непосредственно в защищённой cookie браузера.

Как работает. После проверки логина и пароля сервер подписывает идентификатор пользователя. Браузер автоматически отправляет cookie при следующих запросах. Изменить значение незаметно нельзя без секретного ключа приложения.

```
token = serializer("cookie-auth").dumps({"user_id": user["id"]})
response.set_cookie(
    "auth_cookie", token, max_age=3600,
    httponly=True, secure=HTTPS, samesite="Lax", path="/"
)
```

Критерий	Описание
Достоинства	Минимум серверного состояния; простая реализация; cookie автоматически передаётся браузером.
Недостатки	Сложнее немедленно отозвать уже выданную cookie; размер данных ограничен; требуется защита от CSRF.

Критерий	Описание
Применимость	Небольшие веб-приложения, где достаточно короткой подписанной авторизации.

3.3 Серверная сессия и cookie сессии

Назначение. Хранить состояние входа на сервере, передавая клиенту только случайный идентификатор сессии.

Как работает. Сервер генерирует криптографически случайный token, сохраняет его SHA-256 хеш, пользователя и срок действия в PostgreSQL. Cookie содержит исходный случайный идентификатор. Сессию можно удалить или отозвать на сервере.

```
raw_token = secrets.token_urlsafe(32)
cur.execute(
    "INSERT INTO auth_server_sessions "
    "(token_hash, user_id, expires_at) VALUES (%s, %s, %s)",
    (sha256(raw_token), user["id"], expires),
)
```

Критерий	Описание
Достоинства	Простой отзыв сессий; в браузере нет пользовательских данных; удобна для классических сайтов.
Недостатки	Требуется серверное хранилище и очистка истёкших записей; при распределённой системе нужно общее хранилище сессий.
Применимость	Личные кабинеты, административные панели и серверные веб-приложения.

3.4 Беспарольная аутентификация по электронной почте

Назначение. Выполнить вход без запоминания пароля с помощью одноразовой ссылки.

Как работает. Пользователь указывает электронную почту. Сервер создаёт случайный токен, сохраняет только его хеш и формирует ссылку. В учебном режиме ссылка возвращается в JSON вместо реальной отправки письма. При открытии запись атомарно помечается использованной.

```
UPDATE auth_magic_links
SET used_at = NOW()
WHERE token_hash = %s
AND used_at IS NULL
AND expires_at > NOW()
RETURNING user_id
```

Критерий	Описание
Достоинства	Пользователь не вводит пароль; отсутствует риск повторного использования слабого пароля; удобна для редких входов.
Недостатки	Безопасность зависит от электронной почты; доставка может задерживаться; ссылка должна иметь короткий срок и быть одноразовой.
Применимость	Сервисы с редкими входами, подтверждение операций и восстановление доступа.

3.5 JWT access token

Назначение. Передавать подтверждение личности в API без обращения к хранилищу сессий при каждом запросе.

Как работает. После входа сервер подписывает короткоживущий JWT с идентификатором пользователя, типом access и временем окончания. Клиент отправляет его в заголовке Authorization. Один access-токен допускает несколько запросов до истечения срока.

```
payload = {
    "sub": str(user["id"]),
    "type": "access",
    "iat": now,
    "exp": now + timedelta(minutes=15),
}
token = jwt.encode(payload, JWT_SECRET, algorithm="HS256")
```

Критерий	Описание
Достоинства	Удобен для API и нескольких сервисов; проверяется локально; содержит необходимые claims.
Недостатки	Немедленный отзыв сложнее; содержимое не зашифровано; утечка позволяет действовать до истечения срока.
Применимость	REST API, мобильные приложения и взаимодействие между сервисами.

3.6 Одноразовый JWT refresh token

Назначение. Получить новую пару токенов без повторного ввода пароля и обнаруживать повторное использование refresh-токена.

Как работает. Refresh JWT содержит уникальный jti и действует семь дней. jti хранится в PostgreSQL. При обновлении выполняется атомарный UPDATE только для неиспользованной записи, после чего создаётся новая пара access и refresh. Старый refresh повторно не принимается.

```
UPDATE auth_refresh_tokens
SET used_at = NOW()
WHERE jti = %s
    AND user_id = %s
    AND used_at IS NULL
    AND expires_at > NOW()
RETURNING user_id
```

Критерий	Описание
Достоинства	Короткий срок access-токена снижает последствия утечки; rotation позволяет обнаружить повторное использование; пользователь реже вводит пароль.
Недостатки	Нужно хранить состояние refresh-токенов; требуется безопасное хранение на клиенте и корректный отзыв всей цепочки.
Применимость	Мобильные приложения, SPA и API с длительной пользовательской сессией.

3.7 OAuth2 через единую точку входа

Назначение. Передать проверку пользователя доверенному поставщику идентификации и получить результат через Authorization Code flow.

Как работает. Приложение создаёт случайный state и перенаправляет пользователя в учебную точку входа. Поставщик проверяет логин и пароль, выдаёт короткоживущий одноразовый code и возвращает его на заранее проверенный redirect URI. Callback проверяет state, client_id, redirect_uri, срок и однократность кода.

```
if not secrets.compare_digest(
    state, request.cookies.get("oauth_state", "")
):
    return secure_response({"error": "OAuth state не совпадает"}, 401)
```

```
UPDATE auth_oauth_codes
SET used_at = NOW()
WHERE code_hash = %s AND used_at IS NULL AND expires_at > NOW()
```

Критерий	Описание
Достоинства	Единый вход; приложение не хранит пароль внешней учётной записи; удобно для организаций и экосистем.
Недостатки	Зависимость от поставщика; сложнее настройка redirect URI и ключей; ошибки проверки state создают риск подмены входа.
Применимость	Корпоративный SSO и вход через внешние системы. В реальном проекте можно подключить VK ID, Яндекс ID или собственный сервер идентификации.

3.8 PASETO v4.local

Назначение. Создать безопасный симметрично зашифрованный токен с заранее выбранным современным набором алгоритмов.

Как работает. Из секрета приложения получается 32-байтовый ключ. PySETO создаёт токен версии 4 с назначением local, шифрует claims и добавляет срок действия 15 минут. При проверке библиотека аутентифицирует, расшифровывает и валидирует exp.

```
raw_key = hashlib.sha256(PASETO_SECRET.encode("utf-8")).digest()
key = Key.new(version=4, purpose="local", key=raw_key)
token = pyseto.encode(
    key,
    {"sub": str(user["id"]), "login": user["login"]},
    exp=900,
)
```

Критерий	Описание
Достоинства	Содержимое скрыто; алгоритмы задаются версией протокола; меньше возможностей ошибочно выбрать слабую комбинацию.
Недостатки	Экосистема и совместимость меньше, чем у JWT; секретный ключ должен храниться особенно тщательно.
Применимость	Внутренние API и системы, где требуется конфиденциальность содержимого токена.

4 Сравнение методов

Метод	Состояние на сервере	Отзыв	Основное применение
Подписанная cookie	Не требуется	Ограниченный	Классический небольшой сайт
Серверная сессия	Требуется	Простой	Личный кабинет и админ-панель
Magic link	Одноразовый токен	Простой	Редкий беспарольный вход
JWT access	Не требуется	До истечения сложнее	API и мобильные клиенты
JWT refresh	Хранится jti	Простой	Продление API-сессии
OAuth2	Код и состояние	У поставщика	SSO и внешний вход
PAS ETO local	Не требуется	До истечения сложнее	Зашифрованные токены API

5 Меры безопасности

- Пароли сохраняются только как хеши, созданные `generate_password_hash`.
- SQL-запросы используют параметры и не объединяют пользовательский ввод со значениями SQL.
- Cookie имеют `HttpOnly` и `SameSite=Lax`; `Secure` включается при HTTPS.
- Magic link, refresh-токен и OAuth-код имеют срок действия и используются однократно.
- В базе хранятся SHA-256 хеши случайных session- и magic-токенов.
- OAuth2 использует случайный state и точное сравнение redirect URI.
- JWT проверяются с явно заданным алгоритмом HS256 и обязательным типом токена.
- Ответы получают CSP, X-Content-Type-Options, X-Frame-Options и Referrer-Policy.
- Файл `.env` исключён из Git; в репозитории находится только `.env.example`.

6 Проверка работоспособности

Автоматизированный сценарий test_auth.py запущен против локального сервера и PostgreSQL. Для одноразовых механизмов дополнительно выполнена повторная попытка, которая должна завершиться отказом.

№	Проверка	Ожидаемый результат	Фактический результат
1	Подключение PostgreSQL	База отвечает	Успешно
2	Регистрация	201 Created	Успешно
3	Вход и профиль через cookie	200 OK	Успешно
4	Серверная сессия	200 OK	Успешно
5	Magic link	Первый вход 200	Успешно
6	Повтор magic link	401 Unauthorized	Доступ запрещён
7	JWT access два раза	Оба запроса 200	Успешно
8	JWT refresh	Первое обновление 200	Успешно
9	Повтор JWT refresh	401 Unauthorized	Доступ запрещён
10	OAuth2 Authorization Code	Вход и 200 после redirect	Успешно
11	PASETO v4.local	200 OK	Успешно

7 Вывод

В практической работе реализованы все указанные в задании способы аутентификации и авторизации. Проверка показала, что обычные и токенные способы входа работают, access JWT допускает повторное использование, а magic link и refresh JWT после первого применения отклоняются. Универсального метода для всех систем нет: для классического сайта удобна серверная сессия, для API — короткий access JWT с rotation refresh-токена, для единого входа — OAuth2, а PASETO подходит, когда требуется шифрование содержимого токена и фиксированный современный набор алгоритмов.

8 Используемые источники

1. Flask Documentation. Quickstart and Sessions. <https://flask.palletsprojects.com/en/stable/quickstart/>
2. PyJWT Documentation. Usage Examples. <https://pyjwt.readthedocs.io/en/stable/usage.html>
3. PySETO Documentation. PASETO Usage Examples. <https://pyseto.readthedocs.io/en/latest/>
4. RFC 6749. The OAuth 2.0 Authorization Framework. <https://www.rfc-editor.org/rfc/rfc6749>
5. OWASP Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html